Workshop 2:
Git, GitHub, Quarto, and Website

Red Rock Data Science Conference 2026

GitHub Page: www.github.com/rbrown53/RRDS26

Rick Brown

Southern Utah University

Section 1

Installation Details

Important installations

You will need to install the following:

- **R**: https://cloud.r-project.org.
- RStudio or Positron: https://posit.co/download/rstudio-desktop or https://positron.posit.co/download.html.
- GitHub Desktop Application: https://desktop.github.com/download.

Note: An alternative to GitHub Desktop is to use Git through the terminal. Slides for how to do this can be found on my GitHub page: github.com/rbrown53/RRDS26.

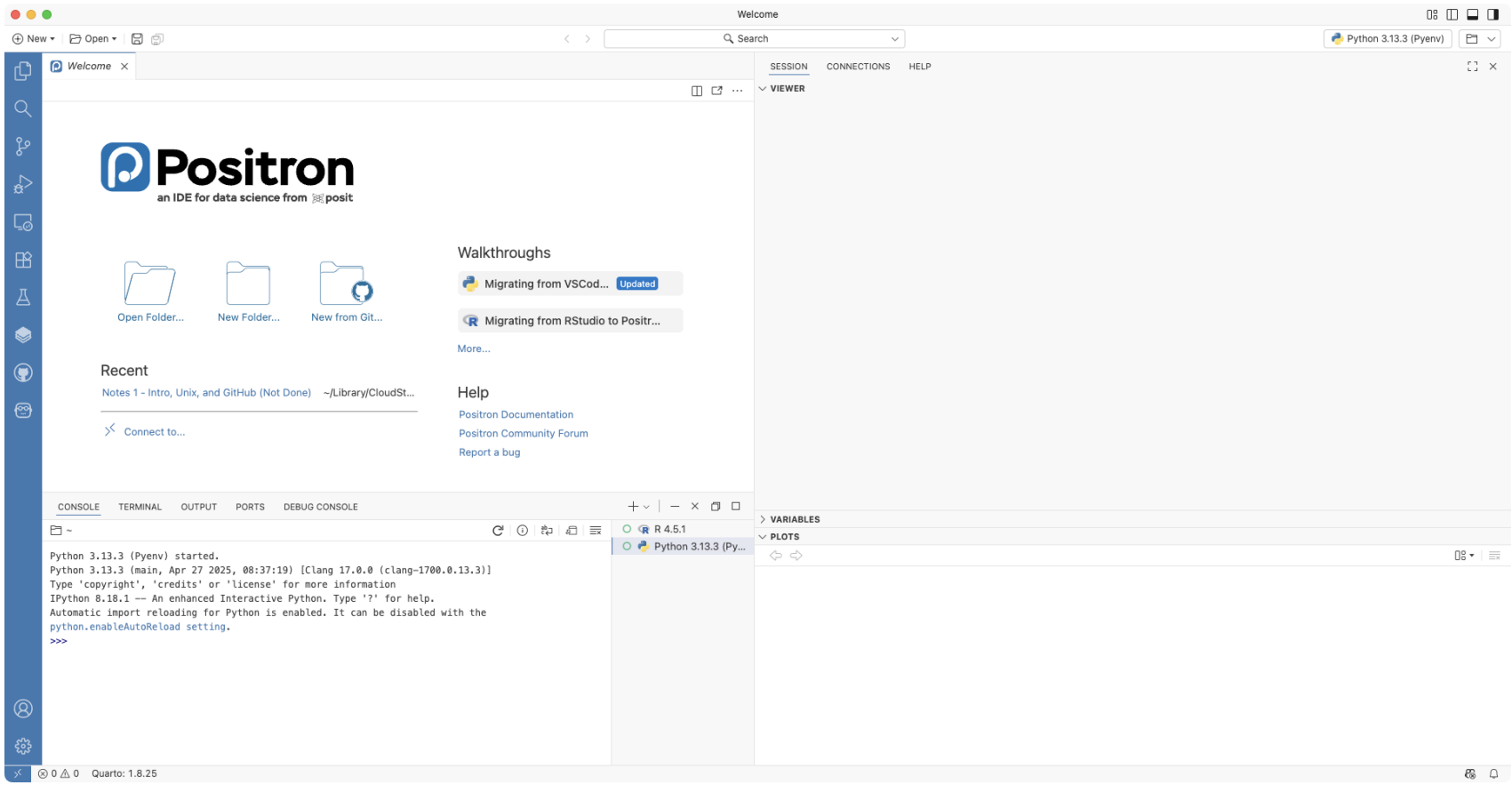
Positron

Positron is a next-generation IDE (Integrated Development Environment) that is designed specifically for data science. It is like a mix between VS Code and RStudio and runs both **R** and Python (among other languages) natively.

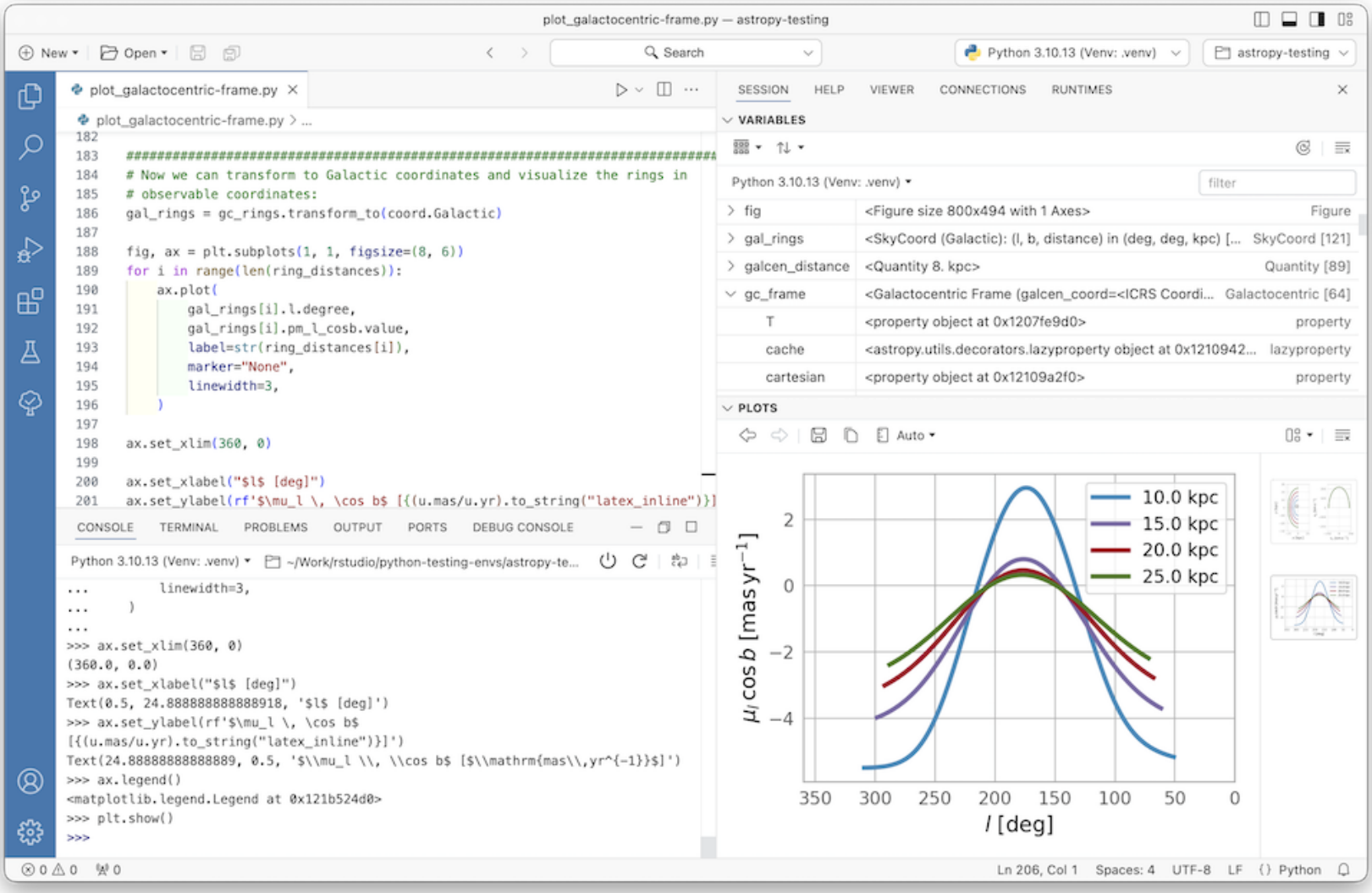
It is developed by Posit, which also created RStudio.

In this class, you can use RStudio or Positron. If you consider yourself primarily a Python programmer, then Positron may be a better fit for you than RStudio. However, I will give instructions is subsequety notes in RStudio, not Positron.

Positron First Impressions



Positron Typical Workflow



Getting Started with Positron

When you first open Positron, it will want you to update the extensions. Click on the Extensions pane on the left side (the one with 4 squares and a red number on it). Then choose to update all extensions.

We will also need to make sure it connects to **R** (and Python if you have that installed). Click on "Start Session" in the top right. You should be able to click on the **R** version you have installed on your computer. If you have Python installed, you an easily switch back and forth between **R** and Python.

Section 2

Git and GitHub

Git and GitHub Introduction

Many software developers and data scientists rely on Git and GitHub everyday.

There are three main reasons to use Git and GitHub.

- Sharing: GitHub allows us to easily share code with or without the advanced version control functionality.
- Collaborating: Multiple people make changes to code and keep versions synced. GitHub also has a special utility, called a **pull request**, that can be used by anybody to suggest changes to your code.
- Version control: The version control capabilities of Git permit us to keep track of changes, revert back to previous versions, and create **branches** in to test out ideas, then decide if we **merge** with the original.

GitHub Accounts

After installing the GitHub Desktop app, you need to create a GitHub account. To do this, go to https://github.com where you will see a link to sign up in the top right corner.

Pick a name carefully! Choose something short, somehow related to your name, and professional. Remember that you will use this to share code with others and you might be sharing this with potential collaborators or future employers!

GitHub Repositories

A GitHub **repository** (repo for short) is like a folder on your computer and allows you to have two copies of your code: one on your computer and one on GitHub. If you add collaborators, then each will have a copy on their computer.

The GitHub copy is usually considered the **main** copy (previously called the **master** and still referred to as master in many tutorials and pages). Git will help you keep all the different copies synced.

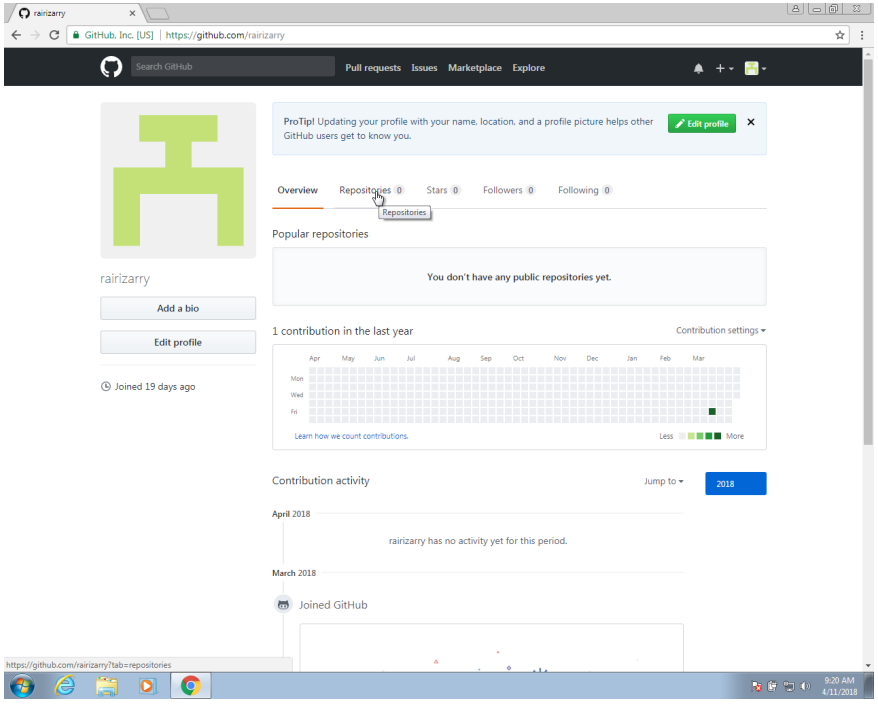
A Note on Privacy

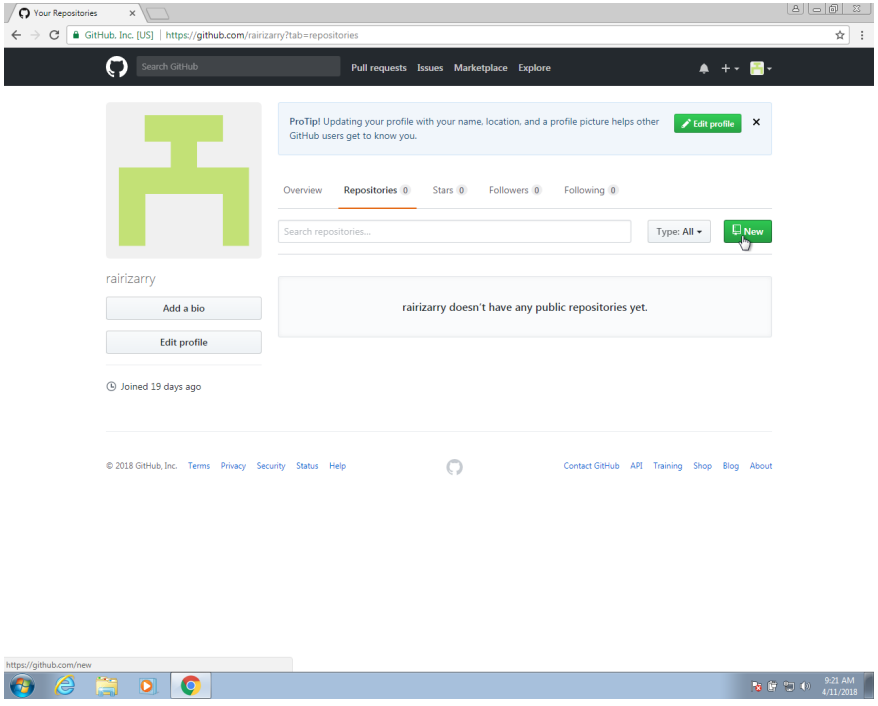
As of April 24, 2026, GitHub will use user data on GitHub to train its AI models by default. If this bothers you, you can opt out of this by doing the following:

- Go to https://github.com/settings/copilot/features
- Sign into your account
- Under Privacy, look for "Allow GitHub to use my data for AI model training"
- Change it from Enabled to Disabled

GitHub Repositories

Now we need to initialize a repository on GitHub. You will have a page on GitHub with the URL: http://github.com/username. On your account, you can click on *Repositories* and then click on *New* to create a new repo:

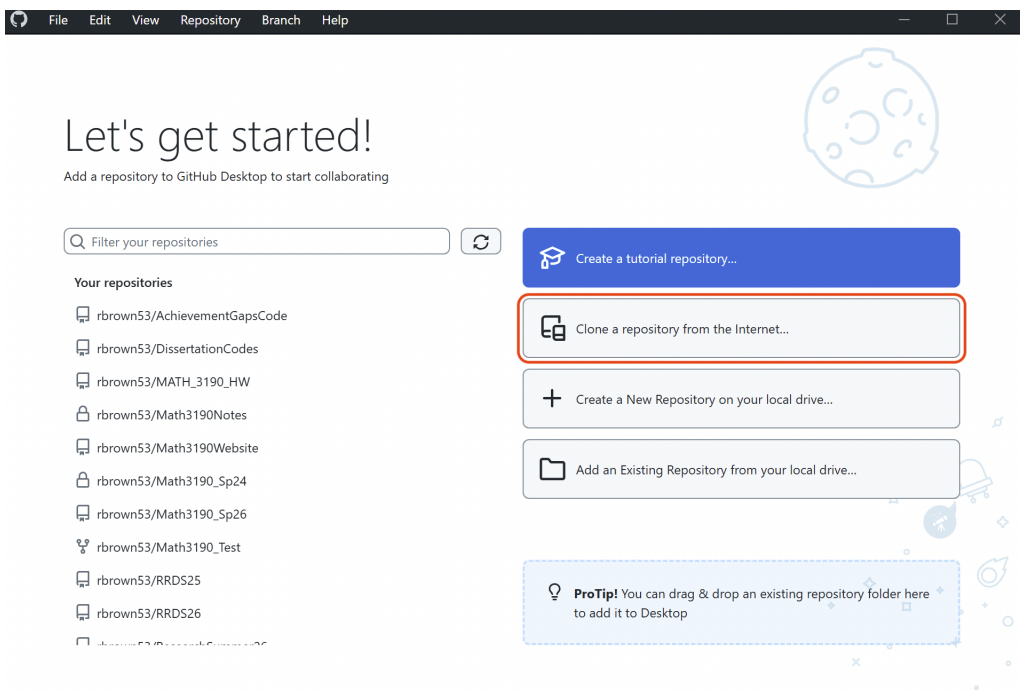


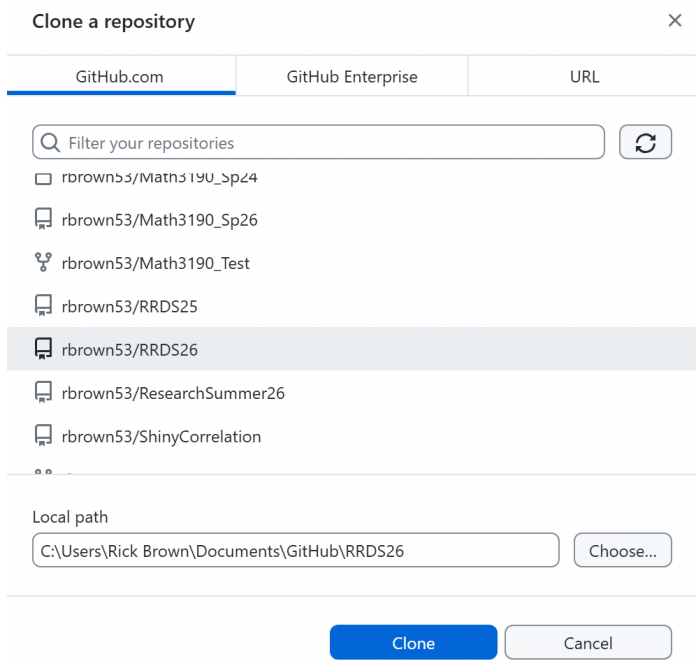


GitHub Repositories

Choose a good descriptive name. In your first activity, you will create one called "RRDS26 GitHub". You don't need to add a `README.md`, a `.gitignore` or choose a license at this time.

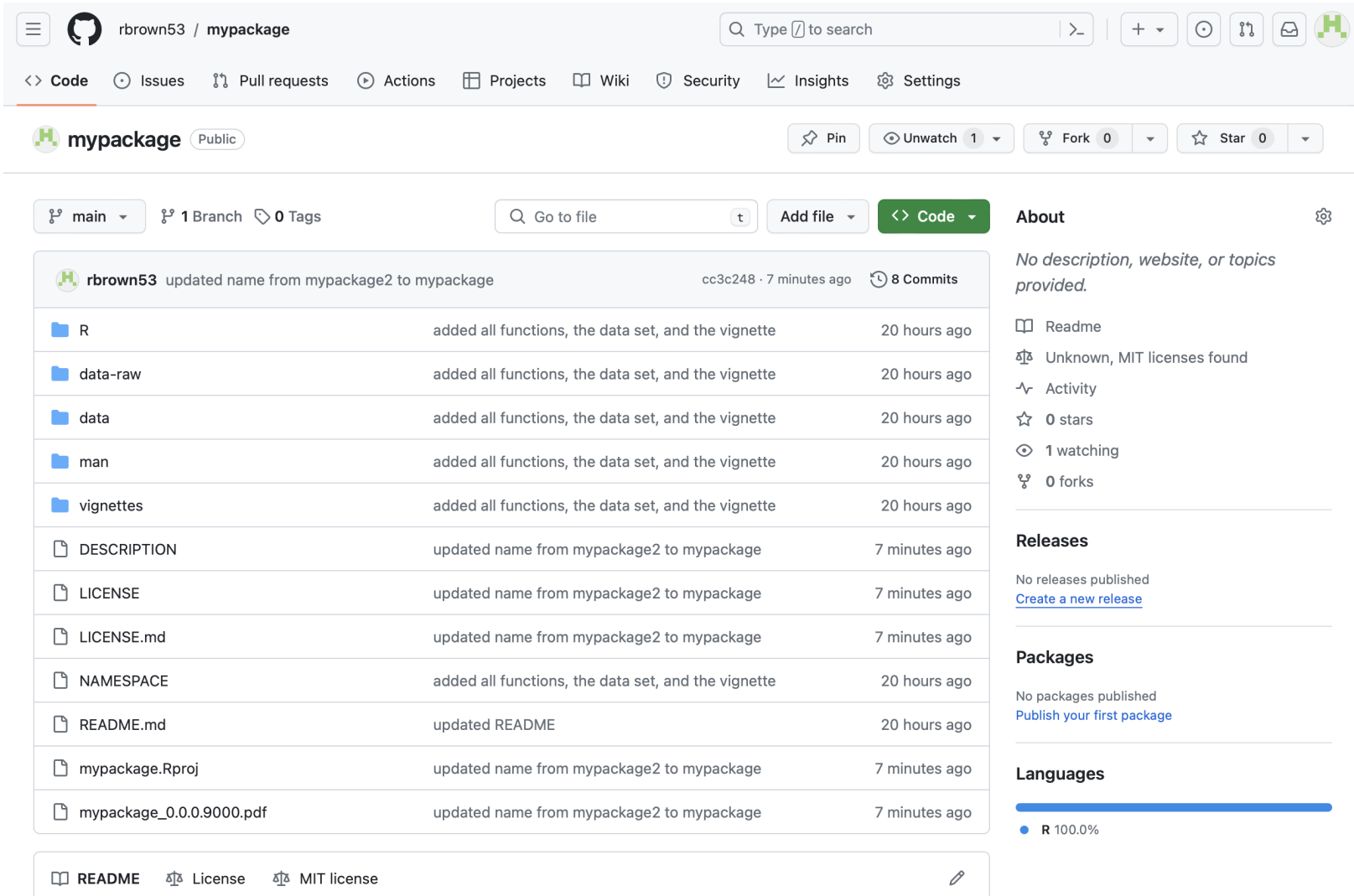
You will then be able to **clone** it on your computer using the GitHub Desktop app.





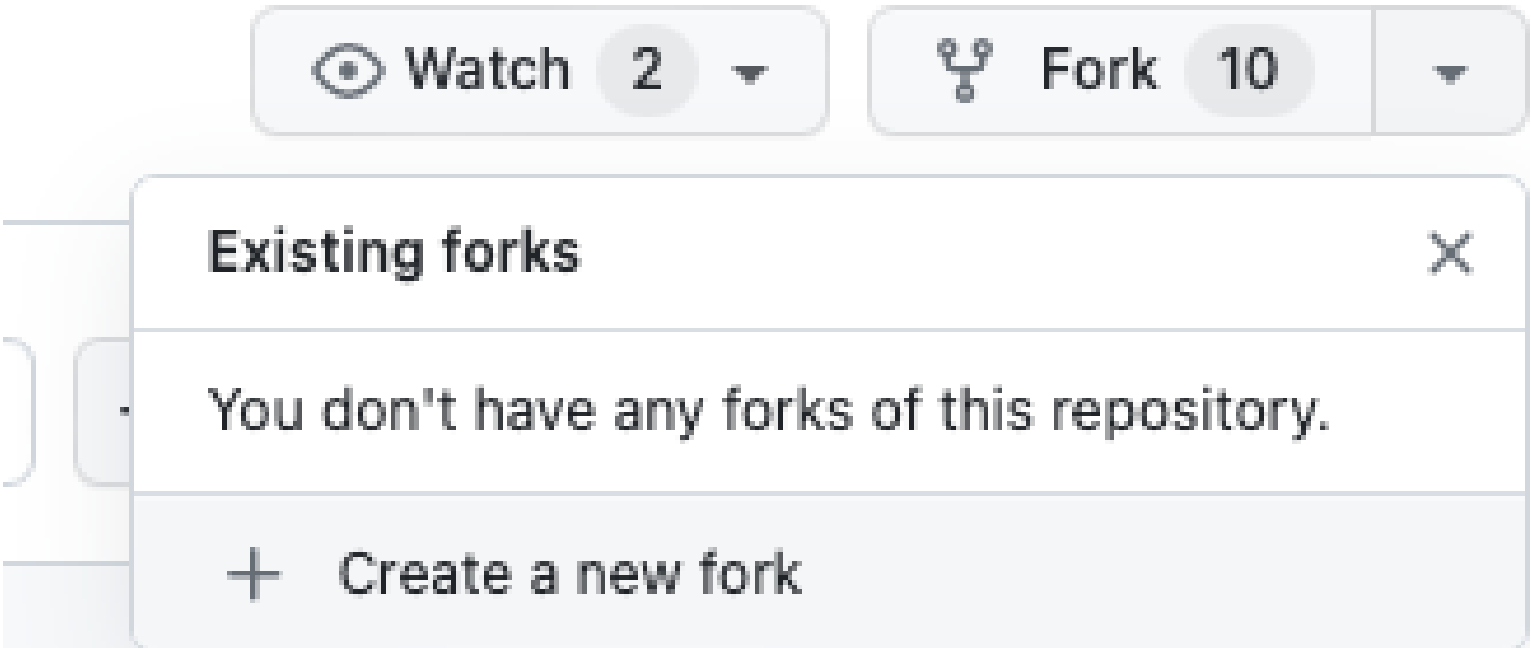
Forking

GitHub also allows you to **fork** others' repos. For example, you could go to https://github.com/rbrown53/mypackage



Forking

Click *Fork* in the top right and this will create a fork of the repository in your GitHub account. This will allow you to alter the code of other people (assuming you are allowed to do so by their license).



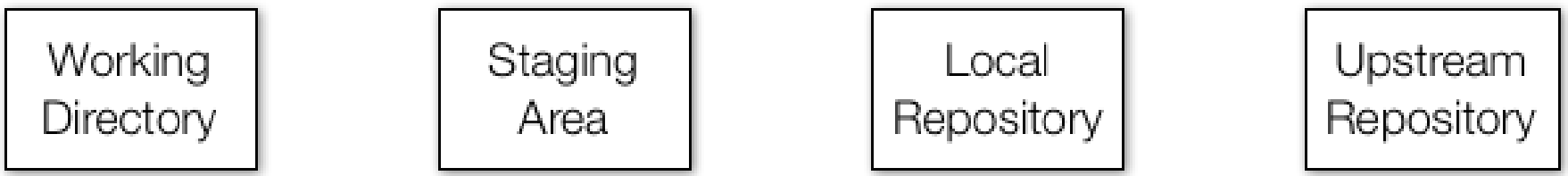
Git Actions

The main actions in Git are to:

1. **pull** changes from the remote GitHub repo.
2. **add** files to the staging area, or as we say in the Git lingo: **stage** files.
3. **commit** changes to the local repository.
4. **push** changes to the **remote** GitHub repo.

Git Setup

To effectively permit version control and collaboration in Git, files move across four different areas:



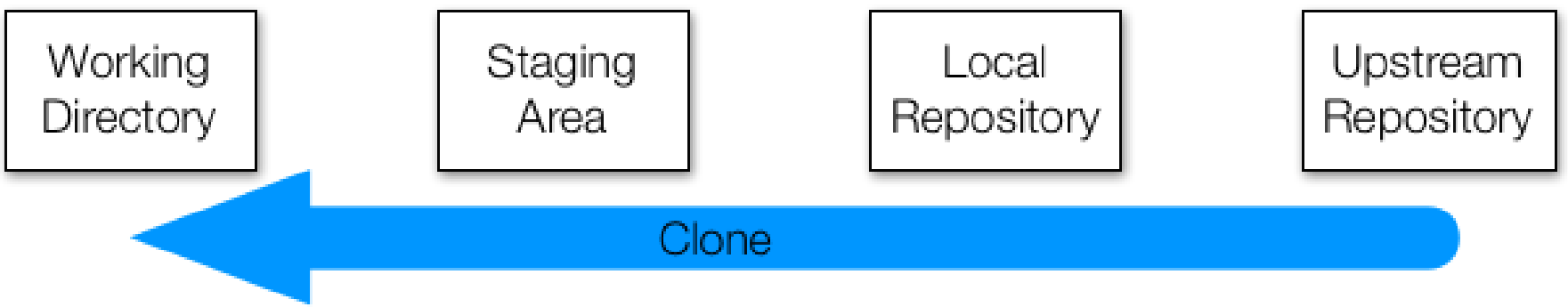
But how does it all get started? We can clone an existing repo or initialize one. We will explore cloning first.

Cloning Git Repositories

You can **clone** an existing **upstream repository** to your local computer.

What does clone mean? We are going to actually copy the entire Git structure, files and directories to all stages: working directory, staging area, and local repository.

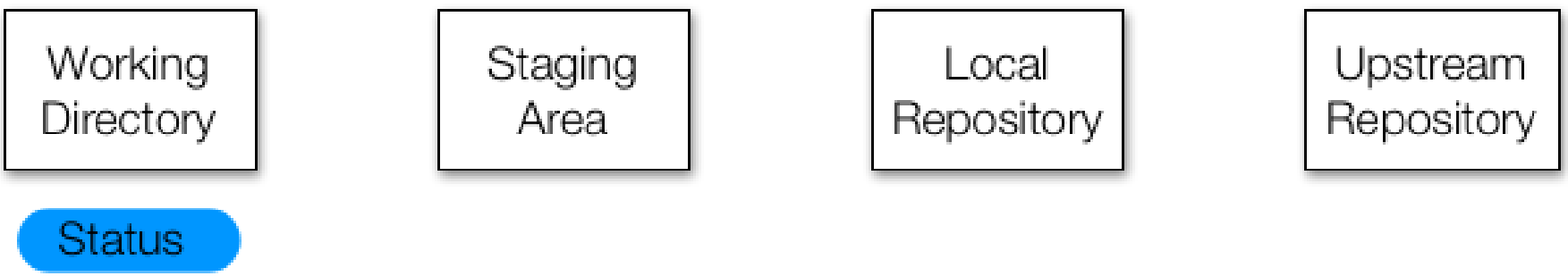
You use the clone feature whenever you initially created a repository online on GitHub or when you fork a repository



Working Directory

Note: the **working directory** is the folder on your computer that you will directly work with. When you edit files (e.g. in RStudio), you change the files in this directory.

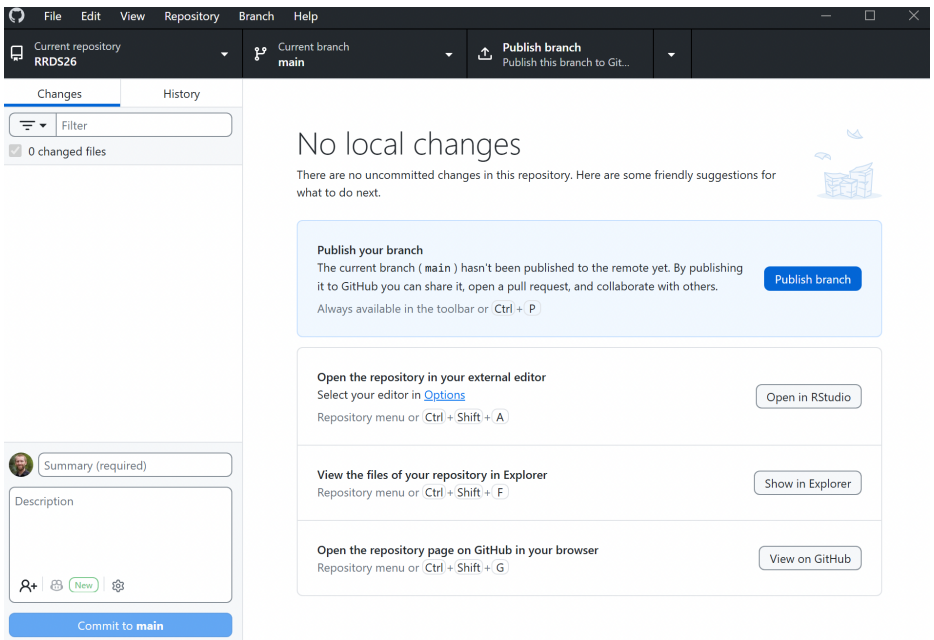
The GitHub Desktop app automatically tracks changes in the working directory.

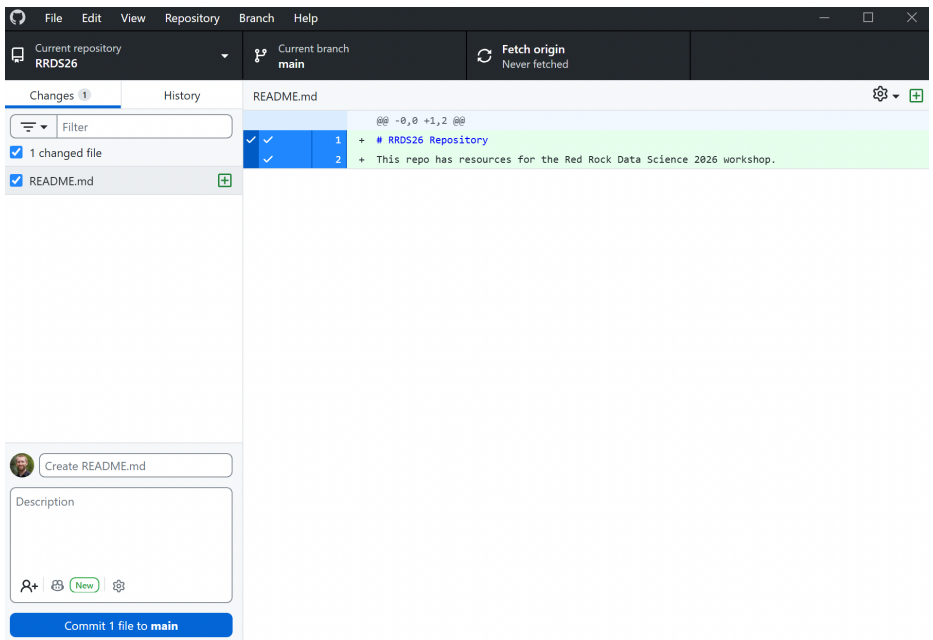


Working Directory

For example, if I have no changes, the page for my repository would look like this image on the left.

If I add a README.md file, I would get this notice in GitHub Desktop shown on the right:



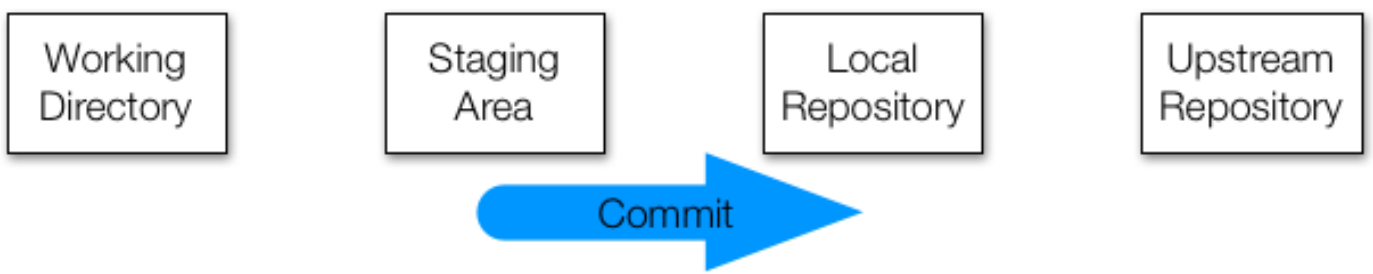


Working with Git Repositories

Now lets **add** the changes to the staging area and **commit** the changes to the local Git directory:

```
git add DESCRIPTION
```


```
git commit -m "updated DESCRIPTION with 'hello' function"
```

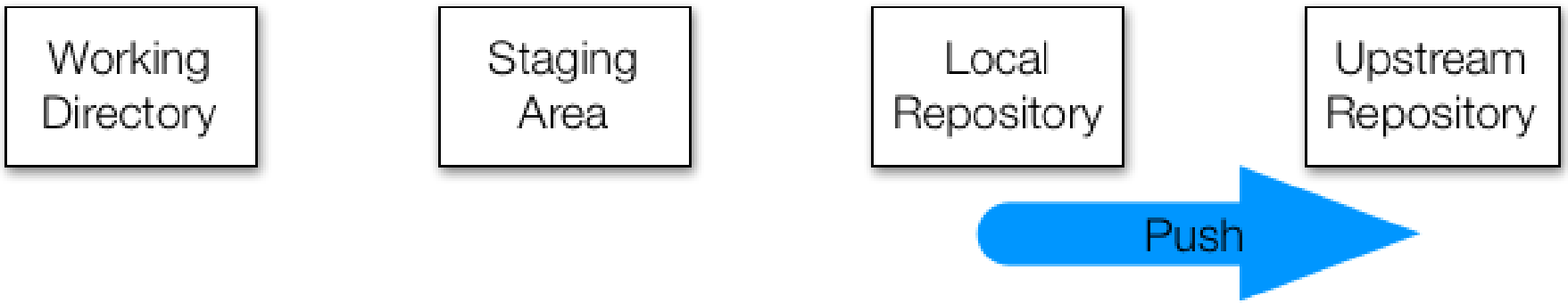


Working with Git Repositories

Now we can **push** the changes to the remote repo:

```
git push
```

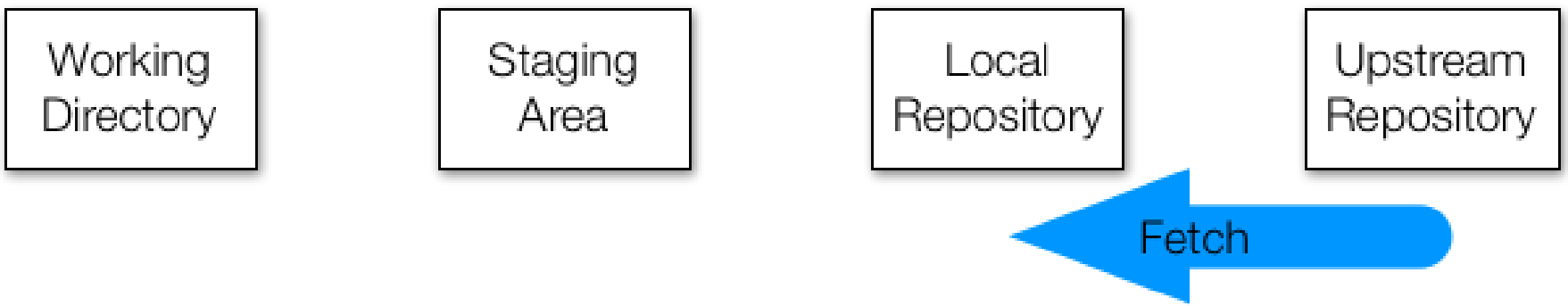
The system will ask for your username and password. The username is your GitHub username, but the password is your personal token that we generated earlier.



Working with Git Repositories

We can also **fetch** any changes on the remote repo (how do you think this is different from `clone`?):

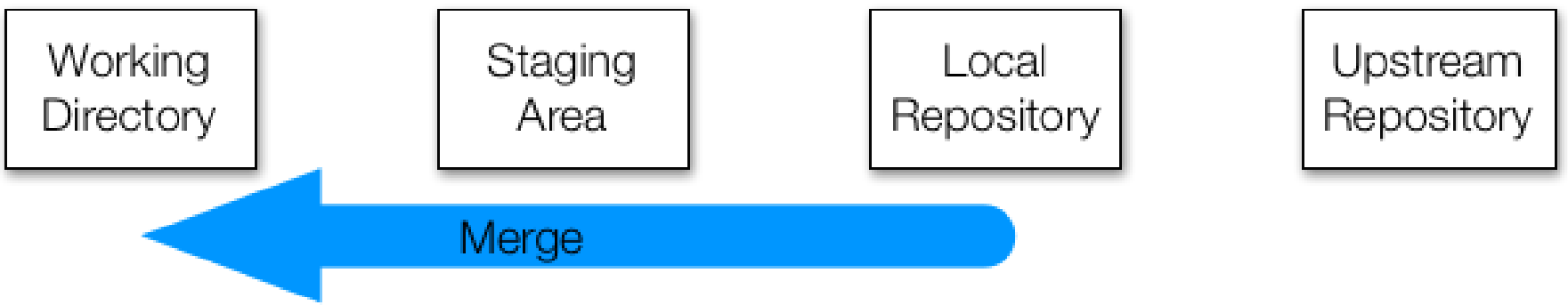
```
git fetch
```



Working with Git Repositories

And then we need to **merge** these changes to our staging and working areas:

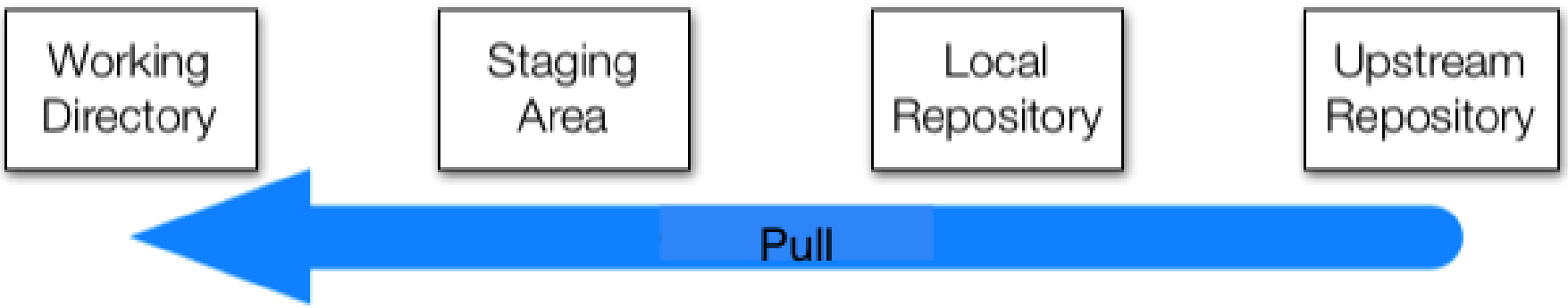
```
git merge
```



Working with Git Repositories

However, we often just want to change both with one command. For this, we use:

```
git pull
```



Important note: it is often a good idea to pull any changes when you start each day, so as to avoid **conflicts**.

Initializing a Git Directory

What if we already have a local directory and want to move it to a GitHub repository? See the following:

1. Create a new GitHub repository (e.g., Math_3190_Assignments)
2. **Initialize** the local repository

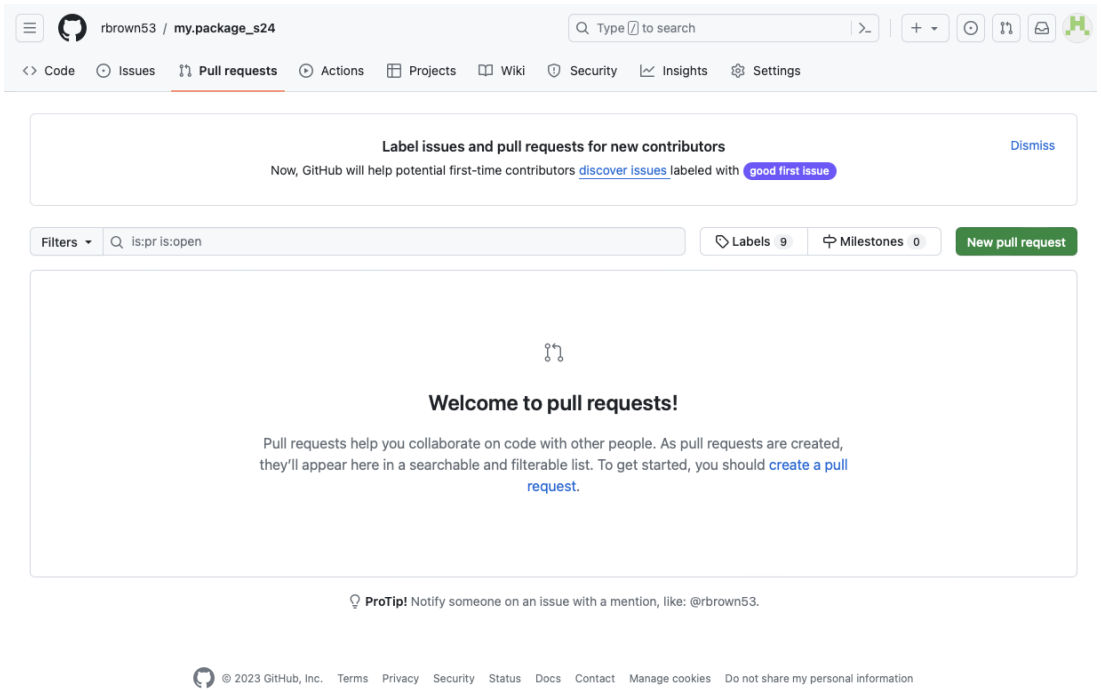
```
git init
```

3. Use the **add** and **commit** commands to add to the local repository
4. Connect the local and remote repos and push:

```
git remote add origin \
https://github.com/username/Math_3190_Assignments.git
git push -u origin main
```

Pull Requests

Pull requests enable sharing of changes from other branches/forks of a repo. Potential changes can be reviewed before they are merged into the base branch. More details on how to do these will be provided in the lab.



Extra - .gitignore File

When we initialize Git in a folder, a hidden folder called `.git` is created. Another hidden file that is useful is `.gitignore`, which lets Git know what files it should NOT keep track of. We can add this when creating a repo on GitHub or we can add this on the local machine by navigating to the directory in the terminal and typing

```
touch .gitignore
```

Extra - Hidden Files

These file and folders that begin with a period (.) are hidden by default, but sometime we want to edit them, especially the `.gitignore`. We can see hidden files by doing the following:

Mac: In Finder, type `Shift-Command-.` (period). Type that again to hide the files again.

Windows 10:

1. Open File Explorer from the taskbar.
2. Select View > Options > Change folder and search options.
3. Select the View tab and, in Advanced settings, select Show hidden files, folders, and drives and OK.

Windows 11:

1. Open File Explorer from the taskbar.
2. Select View > Show > Hidden items.

Conclusion

Positron also has a more visual version control that uses Git built in. It is on the left side and its symbol looks like a branching path.

On Canvas, you'll find a file called "Unix and Git Cheat Sheet.pdf" that contains all of the Git and Unix commands that are contained in these notes plus more.

Section 3

Quarto

Reproducible Reports

The final product of a data analysis project is often a report: scientific publications, news articles, an analysis report for your company, or lecture notes for a class.

Now imagine after you are done you realize you:

- had the wrong data set
- have a new data set for the same analysis
- made a mistake and fix the error, or
- your boss or someone you are training wants to see the code and be able to reproduce the results

Situations like these are common for a data scientist.

Quarto

Quarto is a format for **literate programming** documents.

It is based on **markdown**, a markup language that is widely used to generate html pages.

You can learn more about markdown here:
https://www.markdowntutorial.com/

Quarto is similar to environments like Jupityer Notebooks, Google Colab, and R Markdown (R Markdown is very much like Quarto, but is dependent on **R**). However, Quarto is language agnostic. This means it natively works in both **R** and Python (and Julia and Observable JavaScript).

You can download Quarto here: https://quarto.org/docs/download/.

Quarto Setup

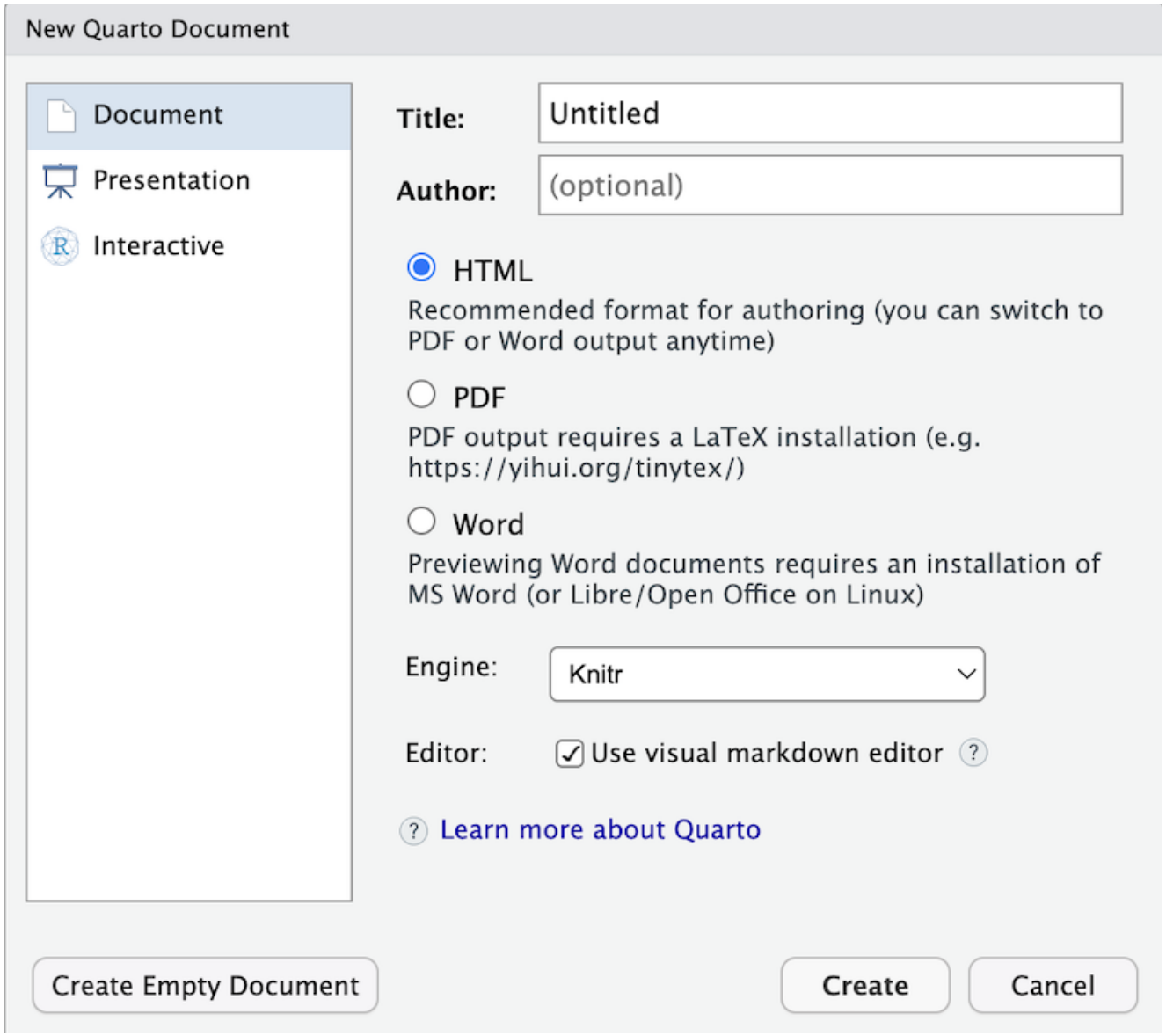
Literate programming weaves instructions, documentation, and detailed comments in between machine executable code.

With Quarto, you need to **compile** the document into the final report. This allows the code to run in a clean workspace each time, so it is reproduceable and gives the same results each time.

You can start a Quarto document in Positron by clicking on **New** then **Quarto Document**. You will then be asked for a title and author.

Final reports can be to be in: HTML, PDF, Microsoft Word, or presentation formats like PowerPoint, Beamer, or HTML slides.

Quarto Setup



Quarto Setup

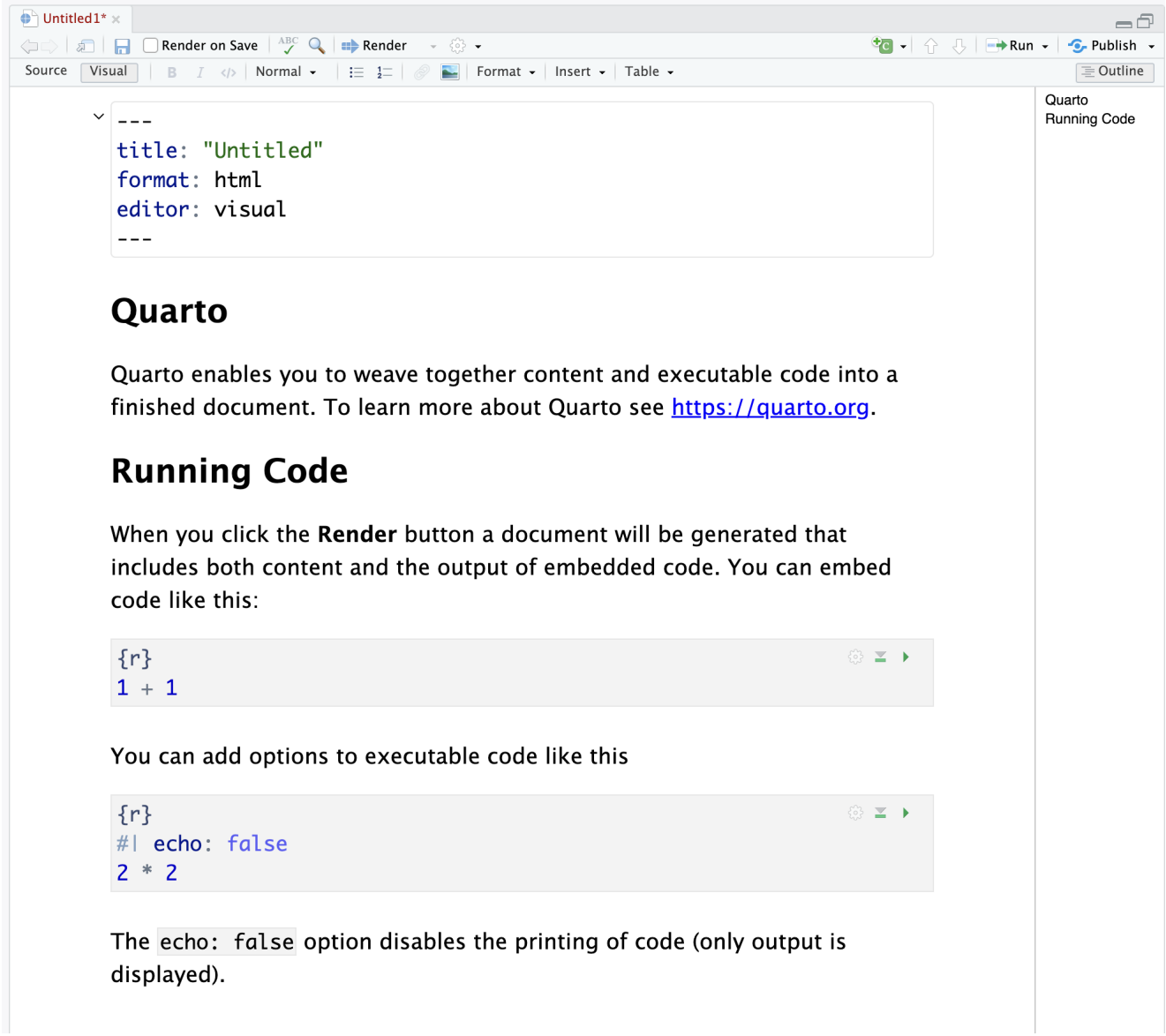
As a convention, we use the `.qmd` extension for these files.

Once you gain experience with Quarto, you will be able to do this without the template and can simply start from a blank template.

In the template, you will see several things to note (in the following slides).

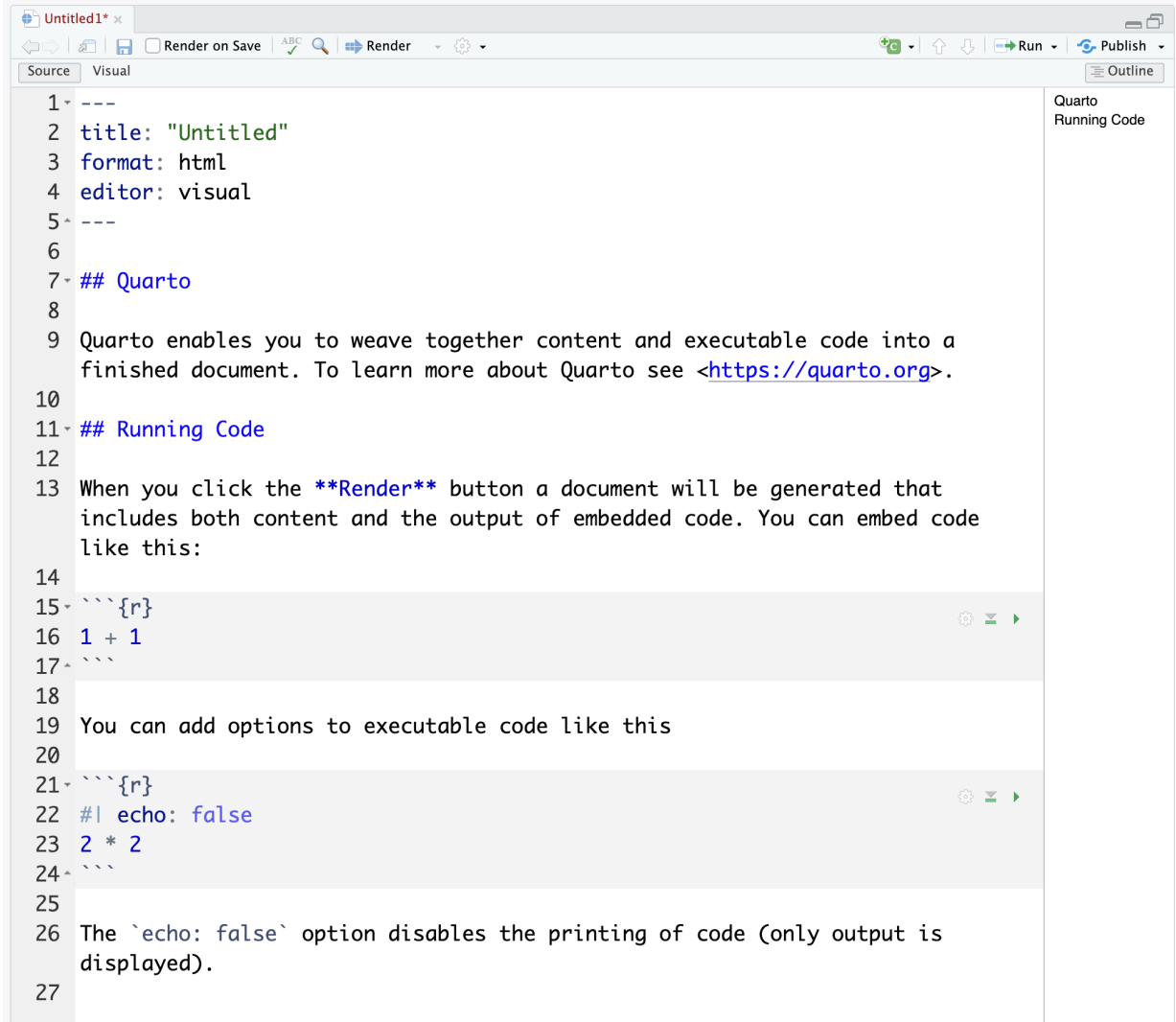
First, the default when opening a Quarto document is that it is in **Visual** editing mode. This will likely be a little more comfortable for you if you are not used to coding in something like LaTeX.

Quarto Visual



Quarto Source

However, we will focus on the source editor since it is much more customizable (although you are not prohibited from using the visual editor).



The Header

At the top of either the visual or source editor, you will see the following:

```
---
title: "Untitled"
format: html
editor: visual
---
```

This is known as the YAML header. YAML stands for YAML Ain't Markup Language.

One parameter that we will highlight is `format`. By changing this to, say, `pdf` or `docx`, we can control the type of output that is produced.

The Header

There are many, many options that can be adjusted in the YAML.

```yaml
---
title: "Project Title"
author: "Your Name"
date: "1/1/2026"
format:
  html:
    code_folding: hide
    toc: true
    theme: "flatly"
editor: source # Makes the source editor the default
               #   when file is opened.
# Like R, you can put comments in the YAML.
---
```

R Code Chunks

In various places in the document, we see something like this:

````
```{r}
1 + 1
```
````

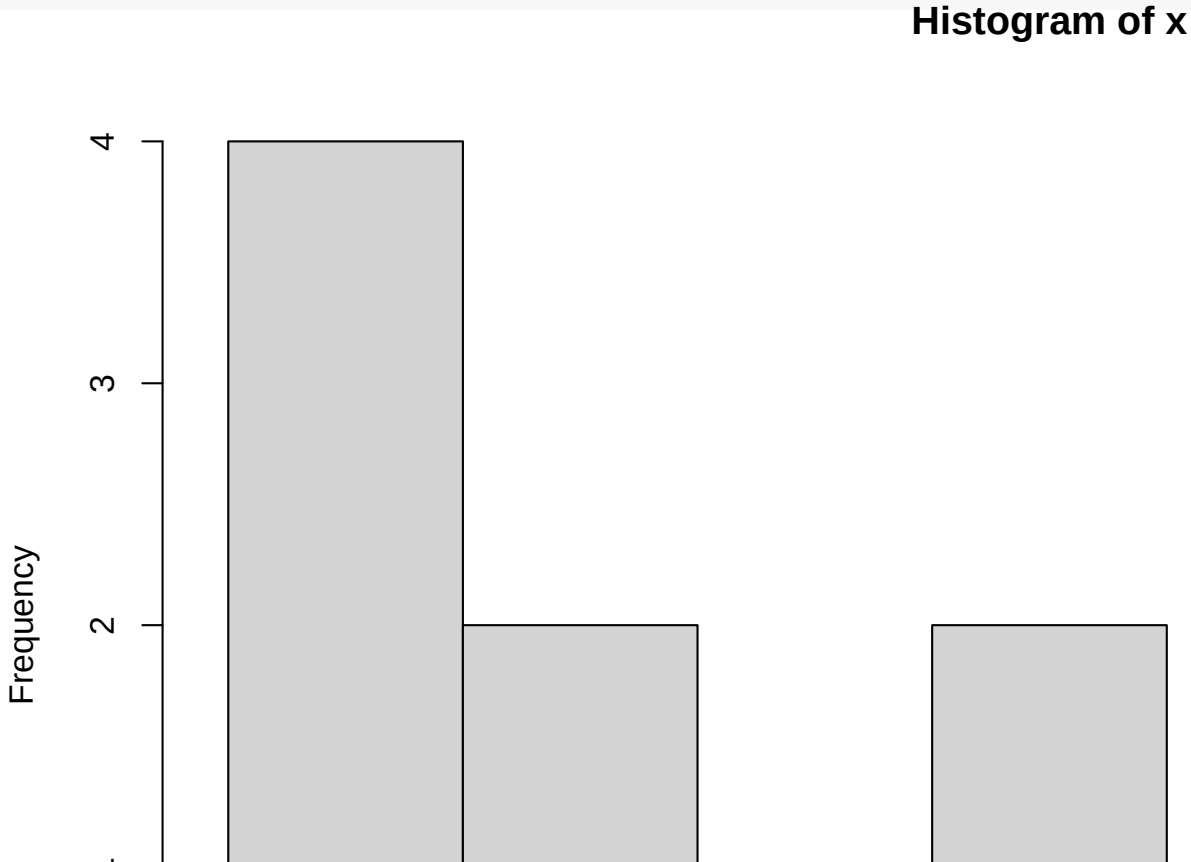
These are the code chunks. When you compile the document, the **R** code inside the chunk, in this case 1+1, will be evaluated and the result included in that position in the final document.

To add your own **R** chunks, you can type the characters above quickly with the key binding Command-Option-I (that is a capital i) on the Mac and Ctrl-Alt-I on Windows. You can also click on the green code chunk button in the toolbar at the top.

R Code Chunks

These code chunks can be used to show code, but that code will also run. Even plots can be made, although this plot is too big to fit on this slide. We will see soon how to adjust the size of plots.

```
x <- c(10, 15, 2, 4, 10, 5, 3, 5, 2)
hist(x)
```



R Code Chunks

By default, both the code and the output will show up in your document. To avoid having the code show up, you can use an argument. To avoid this, you can set options for that code chunk using a special #| comment in the chunk. For example:

````
```{r}
#| echo: false
x <- c(10, 15, 2, 4, 10, 5, 3, 5, 2)
hist(x)
```
````

These comments need to be put at the beginning of the code chunk. Notice that this is YAML, not **R** code. That is why `false` is lowercase while in **R** it would be all capitalized.

R Code Chunks

Its a good habit to add a label to the **R** code chunks. This can be useful when debugging, among other situations. You do this by using the `#| label:` comments:

````
```{r}
#| label: x_hist
#| echo: false
x <- c(10, 15, 2, 4, 10, 5, 3, 5, 2)
hist(x)
```
````

Global Options and knitR

If there are certain options you want to apply to all code chunks, you can specify them in the YAML header with `execute:`. Be aware that white space matters in the YAML. The `echo` and `eval` options need to be indented, need to have a colon, and need to have one space after that colon.

```
---
title: "Untitled"
format: html
execute:
  echo: false
  eval: true
---
```

We use the `knitR` package to compile Quarto documents based on **R**, so we need to install that package. The specific function used to compile is the `knit()` function, which takes a filename as input. RStudio provides a Render button that does that for you, though.

Adding Equations

You can add formatted equations using between dollar signs $. Example, if you type

```
$x^2+\frac{1}{2}$
```

It will look like $x^2 + \frac{1}{2}$. In fact, nearly all LaTeX code works in Quarto.

If you put two dollar signs, the equation will appear on its own line.

```
$$\sum_{i=1}^{10}\left(\lambda_i+\frac{10}{\alpha}\right)$$
```

$$\sum_{i=1}^{10}\left(\lambda_i+\frac{10}{\alpha}\right)$$

Adding and Adjusting Images

We can add images to our Quarto file by using

```
![image_caption_goes_here](image_name.ext){width=50%}
```

if the image is already in the same directory. Otherwise, you can put the path to the file in those parentheses. We can adjust the width of the image as well by changing the number in the `width=50%`.

Adding and Adjusting Images

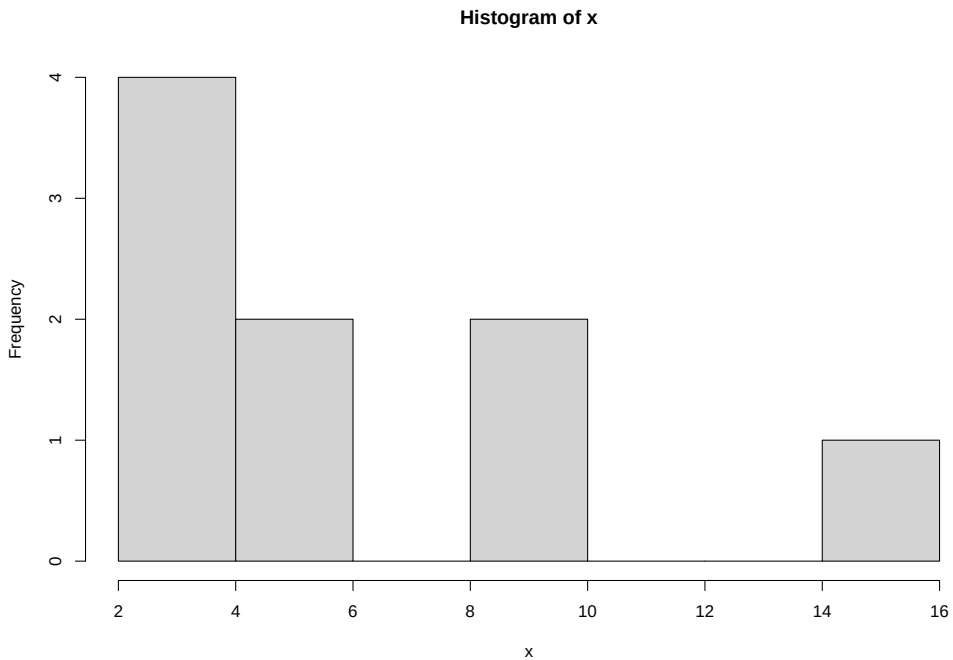
If the image is generated by **R** code, we can adjust the size by using `fig-width`, `fig-height`, `out-width` and/or `out-height` options in the code chunk.

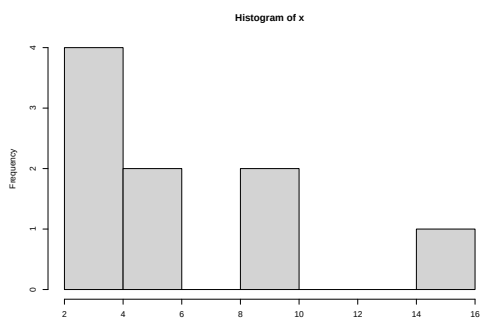
````
```{r}
#| label: image_code
#| label: image_code
#| eval: true
#| echo: true
#| out-width: "25%"
x <- c(10, 15, 2, 4, 10, 5, 3, 5, 2)
hist(x)
```
````

The result is shown othe next slide.

Adding and Adjusting Images

The first image has `out-wdith: "50%"` and the second has `out-wdith: "25%"`





Other Quarto Options

We will explore other Quarto options: headers, tabsets, and latex equations in class and in your homework.

Note: From now on, all homework should be completed using a Quarto document (uploaded to GitHub). Your document should include headers, descriptive text, **R** code, and plots/figures!

More on Quarto

There is a lot more you can do with Quarto. I highly recommend you continue learning as you gain more experience writing reports in **R**. There are many free resources on the internet including:

1. R Studio's tutorial: https://quarto.org
2. The Quarto guide: https://quarto.org/docs/guide/
3. The Quarto definitibe guide: https://bookdown.org/yihui/rmarkdown
4. The knitR book: https://yihui.name/knitr/
5. LaTeX command guide: https://www.bu.edu/math/files/2013/08/LongTeX1.pdf

Extra - Tips

- Make sure you Render your document often. That way you will find any errors quickly without having to search for them.
- Comments in **R** chunks still are the familiar #. But comments in Quarto documents outside of **R** chunks are `<!-- comment here -->`.
- You can bulk comment by highlighting all lines you want commented and then typing `Cmd-Shift-C` on a Mac or `Ctrl-Shift-C` on Windows.

Extra - Mac Quick Look Previewing

If you use the preview feature in macOS (pressing the space bar when highlighting a file in preview), you'll notice that there is no preview available for `.md` files, `.Rmd` files, or files without an extension. You can install QLMarkdown and Syntax Highlight to enable quick viewing of those files.

Section 4

Python in Quarto

Python Intro

One useful feature in Quarto is the ability to run code chunks from other languages. One of the most useful is the ability to run Python inside of Quarto.

There are two ways to do this:

1. Run Python natively.
2. Run Python through **R**.

Run Python in Quarto Natively

If you have Python installed, you can put `engine: jupyter` or `engine: jupyter3` in the YAML. Then you can put Python code chunks like this:

````
```{python}
import numpy as np
np.array([[1, 2, 3], [4, 5, 6]])
```
````

Note that **R** code chunks won't work when using the `jupyter` engine unless other options are specified.

Run Python in Quarto through **R**

We can run Python in Quarto through **R** using the `reticulate` package. This will make it a bit slower than running Python natively.

```
install.packages("reticulate")
library(reticulate)
```

Installing Python and Python Packages

To install Python, we can run the `install_python()` function from the `reticulate` package. Just indicate which Python version you want to install. As of the time these notes were created, the latest version is 3.12.1, but you can look up the latest version or go with the default.

```
install_python("3.14.0")
```

This can take a while to install. Please be patient!

To install Python packages, we need to use an **R** code chunk using the `py_install()` function.

```
py_install(packages = "matplotlib")
py_install(packages = "numpy")
py_install(packages = "pandas")
```

Running Python

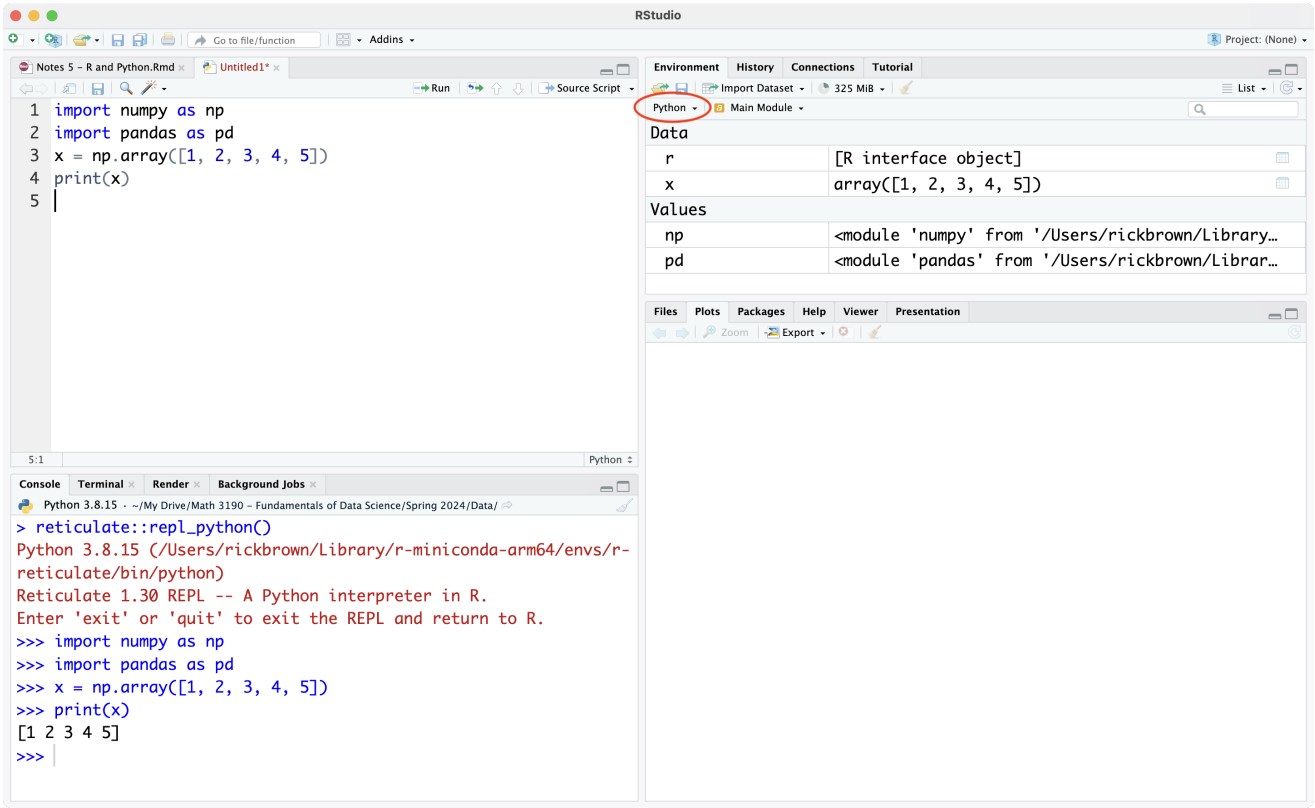
Once we have installed Python, we can insert a Python code chunk just like an **R** code chunk:

````
```{python}
import numpy as np
import pandas as pd
x_python = np.array([1, 2, 3, 4, 5])
print(x_python)
```
````

```
[1 2 3 4 5]
```

Python Envirnoment

When running Python in RStudio, the arrows in the console change and the environment switched to a Python environment. We can view what objects are stored in the **R** and Python envirnoments by clicking on the dropdown button.



Switch Between Python and **R**

We can switch from an **R** enviornment (indicated by a single >) to a Python one (indicated by three arrows >>>) by

- Running a Python code chunk.
- Running the line `reticulate::repl_python()`.
- Double clicking on a Python object in the Python environment list.
- Clicking on the **R** icon at the top left of the Console to bring up a menu and then select Python.

We can switch from a Python environment back to **R** by

- Running an **R** code chunk.
- Typing `exit`.
- Clicking on the Python icon at the top left of the Console to bring up a menu and then select **R**.

Using Python Objects in **R** and Vise-Versa

We can send objects from Python to **R** and vise-versa. Everything defined in Python is stored in the `py` object, which can be called from **R** using `py$name_of_object`. To use an **R** object in Python, we can use preface it with `r.`, so it would be `r.name_of_object`.

```
# R Code. x_python was defined earlier.
library(reticulate)
py$x_python
```

```
[1] 1 2 3 4 5
```

```
y = 2 * py$x_python # Still R code
```

```
# Python Code
print(r.y)
```

```
[ 2.  4.  6.  8. 10.]
```

Type Conversions

While many objects are stored similarly in **R** and Python, it is good to know how things are stored when they are passed back and forth. This table shows the conversion:

R	**Python**
Single-element vector	Scalar
Multi-element vector	List
List of multiple types	Tuple
Named list	Dict
Matrix/Array	NumPy ndarray
Data Frame	Pandas DataFrame
Function	Python function
Raw	Python bytearray
NULL, TRUE, FALSE	None, True, False

Session info

```
sessionInfo()
```

```
R version 4.6.0 (2026-04-24)
Platform: aarch64-apple-darwin23
Running under: macOS Tahoe 26.4.1

Matrix products: default
BLAS:   /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/libBL
LAPACK: /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRlapack.dylib;  LAPACK version 3.12.1

locale:
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8

time zone: America/Denver
tzcode source: internal

attached base packages:
[1] stats     graphics  grDevices utils     datasets  methods   base

other attached packages:
[1] reticulate_1.46.0

loaded via a namespace (and not attached):
 [1] digest_0.6.39     fastmap_1.2.0     xfun_0.57         Matrix_1.7-5
 [5] lattice_0.22-9    knitr_1.51        htmltools_0.5.9   png_0.1-9
 [9] rmarkdown_2.31    tinytex_0.59      cli_3.6.6         grid_4.6.0
[13] withr_3.0.2       compiler_4.6.0    rprojroot_2.1.1   here_1.0.2
[17] rstudioapi_0.18.0 tools_4.6.0       evaluate_1.0.5    Rcpp_1.1.1-1.1
[21] yaml_2.3.12       otel_0.2.0        rlang_1.2.0       jsonlite_2.0.0
```